IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 14 October 2025, accepted 11 November 2025, date of publication 19 November 2025,
date of current version 1 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3634800

RESEARCH ARTICLE

Efficient TSP-Based Task Group Allocation for Multi-Task Multi-Agent Pickup and Delivery

SEUNGBEEN LEE[1], CHANYOUNG LEE[1], WONPIL YU[2], AND SOOHWAN SONG[1]

[1]Department of Computer Science and Artificial Intelligence, Dongguk University, Seoul 04620, Republic of Korea
[2]Artificial Intelligence Creative Research Laboratory, ETRI, Daejeon 34129, Republic of Korea

Corresponding author: Soohwan Song (songsh@dongguk.edu)

This work was supported by the Ministry of Trade, Industry and Energy (MOTIE, South Korea), through the Robot and Robot Service Digital Twinization Framework Technology, under Grant RS-2024-00424974.

ABSTRACT This study addresses the multi-agent pickup and delivery (MAPD) problem, which involves assigning tasks and planning paths for multiple robots to transport goods. Specifically, we focus on the multi-task MAPD where each task consists of multiple targets, and robots are assigned multiple tasks within a budget to deliver in one trip. This problem is challenging because it involves considering an optimal visitation sequence of targets during task assignment. Additionally, managing tasks that are continuously released online demands an efficient algorithm. Therefore, we propose a new task allocation method that iteratively identifies tasks with the shortest detour paths using the TSP algorithm. This method consistently ensures that task groups have the most efficient travel routes for tasks released online. Additionally, we boost computational efficiency by addressing an online TSP, speeding up the computation by focusing on the visitation sequence of only new targets instead of all targets. Extensive experiments on benchmark scenarios demonstrate that our method outperforms existing state-of-the-art approaches, achieving over a 31% reduction in service time.

INDEX TERMS Multi-agent pickup and delivery, task allocation, multi-agent pathfinding, multi-robot system, logistics automation.

I. INTRODUCTION

Automated warehouse systems utilize numerous mobile robots to manage incoming logistic tasks [1], [2], [3], [4], [5]. These robots navigate the warehouse to ensure the timely delivery of goods or inventory pods to designated target sites. For effective logistics management, a robot management system must not only allocate suitable tasks to each robot but also plan collision-free paths for all robots. This challenge forms the basis of the *multi-agent pickup and delivery* (MAPD) problem, extensively studied in recent research [6], [7], [8], [9], [10]. In MAPD, robots are tasked with visiting two specific locations: a pickup location where an inventory shelf is stored and a delivery location within a workstation for the picked item. To execute a task, a robot first travels to the pickup location and then to the delivery location.

The scope of MAPD has recently expanded, with some studies [11], [12], [13], [14], [15], [16] exploring order-picking scenarios where robots gather multiple items for delivery, not just one. This adaptation is termed the *multi-goal MAPD* (MG-MAPD) problem, characterized by tasks involving a series of multiple target locations. Thus, when assigning tasks and planning paths, it becomes crucial to consider routes that cover multiple destinations. Building on this, the *multi-task MAPD* (MT-MAPD) problem was introduced [17], where each robot can carry items up to a specified capacity and handle several multi-goal tasks simultaneously. As shown in Fig. 1, instead of a single task, a group of tasks – referred to as a *task group* – is assigned to a robot, requiring comprehensive path planning to visit all designated target locations across multiple tasks.

The associate editor coordinating the review of this manuscript and approving it for publication was Jenny Mahoney.

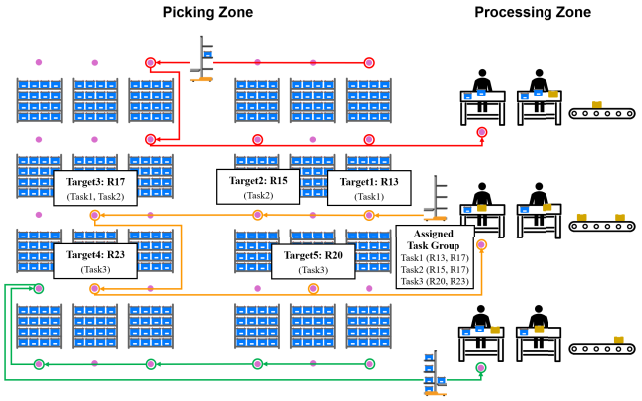

FIGURE 1. Illustration of multi-task MAPD problem. Multiple agents navigate rack-filled areas to pick up items and deliver them to designated zones where workers receive them. In multi-task MAPD, each robot can carry items up to a certain capacity and manage multiple tasks with several targets, grouped into a task group.

In this study, we explore the MT-MAPD, the most general version of the MAPD problem. MT-MAPD diverges from the traditional format by not having a fixed sequence for visiting targets. Therefore, MT-MAPD increases complexity as the most efficient sequence must be considered when assigning tasks. Each time a new task is added to a task group, the sequence of target visits needs updating. Moreover, despite the problem's complexity, an efficient online algorithm is essential for managing continuously released tasks. The TSP algorithm, used to determine visitation order, significantly adds to the computational load, necessitating an effective alternative. Additionally, computing task assignments in parallel during the robots' movement phases could save significant time. However, current methods [6], [11], [12], [13], [17] have not explored this potential, making them less applicable in real-world settings.

To address these challenges, we propose a novel task allocation algorithm for MT-MAPD designed to generate task groups that ensure minimal travel distances for robots. This algorithm leverages the TSP to calculate the shortest path that visits all target locations within a task group. It iteratively finds a task that causes the smallest path extension when detouring to new target locations, adding it to a task group until the robot's budget limit is reached. This approach ensures the inclusion of tasks based on the proximity of their targets, facilitating the formation of a task group that navigates all targets via the shortest path. The algorithm also continuously replaces previously assigned tasks with newly released ones if they offer shorter detour paths. This approach allows for the maintenance of optimal task groups as new tasks released online. This differs from previous methods [6], [7] that merely consider proximity to a single target, as it takes into account the entire travel path for all targets. Therefore, the proposed algorithm can significantly improve service time and reduce makespan compared to existing methods [6], [7], [17].

To further enhance computational efficiency, we also developed an efficient version of the task allocation algorithm. Determining the shortest travel sequence with the TSP for every task involves substantial computation. Thus, we employ an approximation algorithm for the TSP operation to quickly compute the shortest path. This method reduces the computation time of the original algorithm by more than 16 times, while still maintaining solution quality to a certain degree. Furthermore, we have also introduced a framework that allows each robot to perform pre-computed task groups sequentially. This framework operates independently from the path planning module, enabling the generation of multiple sequential task groups from released tasks in parallel. This method is particularly effective in real-world scenarios involving actual robots, as it enables continuous optimization of the task list while the robot is moving. This approach provides a higher quality solution than merely planning upon receiving a task allocation request.

The contributions of this study include:

- We propose a novel task allocation algorithm for MT-MAPD, which considers the optimal path for visiting all targets. This method continuously maintains the task groups with the shortest travel path for tasks released online.
- We enhance the computational efficiency of task allocation by addressing an online TSP. This approach speeds up the solution by focusing on computing the visitation sequence only for new targets, instead of all targets.
- Unlike existing methods [6], [12], [13], [17] that integrate task allocation and path planning, the proposed approach solves them in parallel. This allows the task allocation algorithm to provide sufficient time for finding high-quality solutions when robots are moving.
- We evaluate the proposed method through extensive experiments in Kiva warehouse and sorting center scenarios [11]. The experiments show that the proposed method outperforms state-of-the-art methods [6], [17], notably reducing service time by over 31.1%. Source code for the proposed method is publicly available.[1]

II. RELATED WORK

The MAPD is a complex problem that simultaneously tackles task assignment and *multi-agent pathfinding* (MAPF). Task allocation [17], [18], [19], [20] aims to assign tasks to agents in a way that minimizes the total cost of completing all the given tasks. MAPF [21], [22], [23], [24], [25] focuses on finding optimal paths for multiple agents to their designated destinations without causing collisions. MAPD studies are divided into coupled and decoupled methods. Coupled methods [13], [26], [27], [28], [29] integrate task allocation and path planning, achieving better coordination and globally optimal solutions but with higher computational complexity. Decoupled methods [6], [10], [11], [17], [30] address task allocation and path planning as separate problems, solving them independently. This approach simplifies computation and improves scalability, but it may sometimes yield

[1] https://github.com/seungbeeen/TSP-MAPD.git

suboptimal solutions due to limited coordination between the two stages.

This work targets large-scale online MAPD with up to 100 agents, where task assignment and path planning must be repeatedly executed in real time during operation. To meet these runtime and scalability requirements, we adopt a decoupled architecture that separates assignment from routing. In grid-like warehouse layouts with ample detours and maneuvering room, the risk of path overlap and deadlocks is relatively low; consequently, the decoupled approach incurs negligible performance loss compared to coupled formulations while substantially reducing computational and replanning overhead. Table 1 provides a summary of prior studies on the MAPD problem.

Traditionally, the focus has primarily been on the one-shot version of MAPD, also known as a target assignment and path finding (TAPF) problem [26], [27], [29], [30]. The objective of TAPF is to solve a single MAPD instance. Ma and Koenig [26] developed an algorithm that effectively addresses the TAPF problem by grouping agents and goals. They applied a min-cost max-flow algorithm for intra-team task assignments and a conflict-based search (CBS) [31] for resolving inter-team collisions. This method provides optimal solutions but may struggle with scalability in dense environments due to its coupled nature. Honig et al. [27] expanded on this by introducing CBS-TA, a variation of CBS that simultaneously assigns tasks and plans collision-free paths. While providing optimal solutions, the scalability of CBS-TA is also limited. Zhong et al. [29] addressed a multi-goal TAPF problem, where each task involves a sequence of multiple ordered goals. They optimally solve this problem by integrating the CBS-TA with the multi-label A* [10]. Ho et al. [32] presented HTAPPS, a hierarchical approach that decomposes the MAPD problem into three levels of abstraction to reduce computational effort. This method clusters tasks with nearby targets and solves the TSP between clusters instead of all individual targets, allowing for faster computation of the visitation order. Ren et al. [28], [33], [34] extended MAPF to multi-agent combinatorial path finding, where agents must visit intermediate targets before reaching their destinations. To address this complex problem, they proposed Conflict-Based Steiner Search (CBSS), which combines CBS with traveling salesman algorithms to optimize target sequencing and compute efficient paths.

More recent research [6], [7], [8], [9], [10], [35] has shifted towards a lifelong version of MAPD to address real-world challenges across various applications. Ma et al. [6] first defined the online MAPD problem, which solves a continuous stream of MAPD instances for newly released tasks. They proposed a Token Passing (TP) method where a token, signifying authority, is passed among agents. The agent holding the token assigns the nearest task and plans the subsequent path. TP is highly scalable and simple to implement, but its greedy nearest-task-first assignment strategy can lead to inefficient global solutions in certain scenarios. Beyond centralized and token-based approaches, distributed strategies have been explored to enhance robustness. For instance, [36] investigates distributed task assignment under limited communication ranges, which is critical for real-world deployments. However, such methods often trade off global solution quality for improved scalability and resilience. Grenouilleau et al. [10] proposed a multi-label A* algorithm for planning paths covering consecutive locations, introducing a heuristic-based task assignments using h-values of A* search. Liu et al. [7] approached the offline MAPD problem, where all tasks are known initially. They transformed a MAPD instance into a directed graph and computed its Hamiltonian cycle to allocate a task sequence of each agent. This method assumes all tasks are known a priori, limiting its applicability to dynamic online settings.

Studies [11], [12], [13], [38] have also expanded MAPD into MG-MAPD, where tasks involve multiple pickup locations. Li et al. [11] introduced a Rolling-Horizon Collision Resolution (RHCR) method for planning paths aimed at multiple goals. This method updates multi-agent paths by resolving collisions within a limited time horizon. While this method delivers high-quality paths, it does not address task allocation directly. Xu et al. [12] enhanced this approach by incorporating task sequence assignment into the RHCR method. They began with an initial sequence generated using a Hungarian-based insertion for each agent and refined it through continuous Shaw removal [39] and regret-based re-insertion [13]. Although this method is effective, its regret-based reinsertion process is computationally intensive for real-time applications. Chen et al. [13] introduced the Regret and Marginal-Cost Based Task Assignment Algorithm (RMCA). Initially, it applies a standard regret-based marginal-cost heuristic to assign a task sequence to

TABLE 1. Summary of MAPD studies. Studies are classified into "One-shot" and "Lifelong" versions of MAPD based on the dashed line. They are also classified into "Coupled" and "Decoupled" methods. "Multi-Goal" and "Multi-Task" refer to studies addressing the MG-MAPD and MT-MAPD problems, respectively.

Method	One-Shot/ Lifelong	Coupled/ Decoupled	Multi-Goal	Multi-Task
CBM [26]	One-Shot	Coupled	✗	✗
CBS-TA [27]	One-Shot	Coupled	✗	✗
(SMT-)HCBS [30]	One-Shot	Decoupled	✓	✗
CBSS [28]	One-Shot	Coupled	✓	✗
CBS-TA-MLA [29]	One-Shot	Coupled	✓	✗
HTAPPS [32]	One-Shot	Decoupled	✓	✗
TP [6]	Lifelong	Decoupled	✗	✗
HBH+MLA* [10]	Lifelong	Decoupled	✗	✗
TA-Hybrid [7]	Lifelong	Decoupled	✗	✗
RHCR [11]	Lifelong	Decoupled	✓	✗
LNS-PBS [12]	Lifelong	Decoupled	✓	✗
RMCA [13]	Lifelong	Coupled	✓	✗
HATA [32]	Lifelong	Decoupled	✓	✗
TSP-MAPD [17]	Lifelong	Decoupled	✓	✓
TSP-MAPD+ [37]	Lifelong	Decoupled	✓	✓
Ours	Lifelong	Decoupled	✓	✓

each agent. Then, using a greedy heuristic, the algorithm iteratively removes and reassigns a subset of tasks within the sequence. Li and Huang [35] tackled the Task Planning for Heterogeneous Agents (TPHA) problem, where tasks are assigned to different types of agents to minimize total costs, including travel and tardiness costs. They introduced a framework featuring an efficient algorithm for calculating the costs between tasks and heterogeneous agents. In a similar vein, the work in [40] and its extension [41] addresses precedence-constrained delivery for heterogeneous vehicles, focusing on efficient routing under complex task dependencies.

With the rise of diverse logistics transportation scenarios, there has been a growing demand for addressing the more general problem, MT-MAPD. However, few methods currently address MT-MAPD effectively. Kudo and Cai [17] proposed a TSP-based method for MT-MAPD that assigns tasks randomly to agents whose budget limits have not been exceeded. If multiple tasks are assigned to an agent, the TSP algorithm determines the visitation order of all target locations, and the agent is moved accordingly. The major weakness of this approach is its random task allocation, which often leads to poor solution quality as it ignores the spatial efficiency of assignments. They further enhanced the method by tackling the MT-MAPD problem with anytime task allocation under energy constraints [37]. Previously, tasks were assigned to agents starting at their initial positions [17], but this approach [37] assigns tasks to vacant agents with available capacity after completing their previous tasks. Although this improves upon [17], the core issue of non-optimal, random task assignment remains unresolved, fundamentally limiting the solution quality. Additionally, battery levels are taken into account, ensuring tasks are given to agents with sufficient energy to complete them.

In summary, despite a solid foundation, the existing literature still falls short for MT-MAPD: (i) coupled formulations achieve strong coordination but are often computationally prohibitive, and (ii) many decoupled approaches rely on naive (e.g., random or nearest) assignment heuristics that yield suboptimal solutions. We address both issues. Instead of the random allocation used in [17] and [37], our method derives assignments from TSP-based shortest-distance structures over all targets, markedly improving effectiveness. Furthermore, unlike tightly coupled methods that suffer from high complexity, our approach structures task allocation and path planning as separate modules. This decoupled design enables parallel processing, offering a better balance between solution quality and computational efficiency for practical robot operations.

III. PROBLEM FORMULATION

This section describes the MT-MAPD problem, as illustrated in Fig. 1. In this problem, multiple agents navigate spaces filled with racks to load ordered items and then deliver them to designated areas where workers are prepared to receive

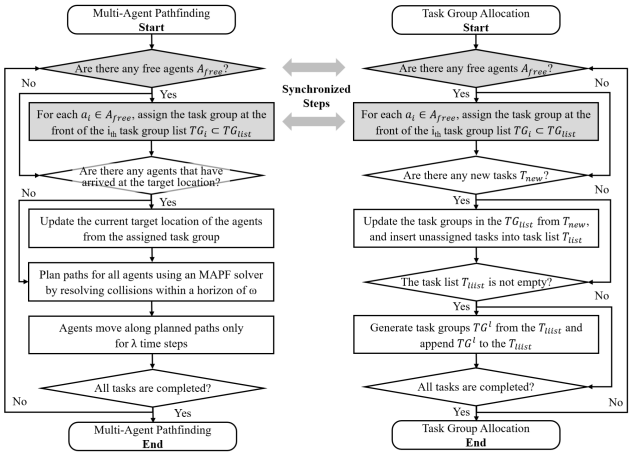

FIGURE 2. Flowchart of the MAPF and task allocation algorithms, organized as separate modules. Gray boxes indicate processes that are synchronized across both modules. The MAPF module directly adopted the RHCR method [11].

them. Let $A = \{a_1, \ldots, a_N\}$ represent a set of agents. Each agent $a_i \in A$ moves on an undirected graph $G = (V, E)$, where each vertex $v \in V$ represents a 2D location and each edge $e \in E$ connects two locations. At every discrete timestep t, an agent a_i can either move to an adjacent location or remain at its current location.

The MT-MAPD problem is divided into two sub-problems: task allocation and MAPF. In the task allocation, a task τ is defined by a set of target locations $\{v_1, \ldots, v_K\}$ and a release time t_{rel}. The system iteratively releases new tasks based on their release times. Task group, denoted as $TG = \{\tau_1, \ldots, \tau_M\}$, is a collection of such tasks. In MT-MAPD, each agent is assigned a task group rather than a single task. Agents must adhere to payload capacity limits, which constrain the number of targets in a task group so as not to exceed this capacity. Let V_{target} be the collective target locations of all tasks within a task group. To complete the task group, the assigned agent must collect items from each location in V_{target} and transport them to a delivery location. An agent that has finished its assigned task group is referred to as a "free agent," and a new task group is promptly allocated to such free agents. The objective is to minimize the overall time taken to complete tasks, known as the *makespan*, or to decrease the *average service time*, which is the average difference between a task's completion and release times.

For the MAPF part, each agent must plan a collision-free path that covers all the targets in its assigned task group. The path p_i for an agent a_i is defined as a sequence of vertices on G. Collisions occur between two paths p_i and p_j when agents a_i and a_j arrive at the same vertex simultaneously, resulting in a vertex collision, or when they traverse the same edge in opposite directions at the same time, leading to an edge collision [31]. The goal of a MAPF instance is to find a set of paths $P = \{p_1, \ldots, p_N\}$, that are free of collisions and minimize the total travel time. As task groups of

agents are updated, the system continually solves the MAPF instance.

IV. MULTI-TASK MULTI-AGENT PICKUP AND DELIVERY

Most prior studies [6], [11], [12], [13], [17] have proposed integrated algorithms for the MAPD problem, addressing task allocation and MAPF as separate processes in a sequential manner. These algorithms typically perform task allocation and path planning sequentially, either when an agent becomes free or upon receiving a task assignment request. In online scenarios, where solutions must be computed in real-time, most algorithms prioritize efficiency over the optimality of a solution. Consider the scenario of operating real robots. In such cases, an agent with an updated task needs to plan its path immediately. If the agent's starting position changes during the planning process, the planned path will no longer be applicable. Thus, MAPF necessitates an efficient algorithm that supports real-time processing. For task allocation, unlike path planning, there is no urgency for immediate computation; tasks can be pre-determined and allocated in a sequential manner. Additionally, the time it takes for robots to physically move provides an opportunity to find more optimized solutions.

Therefore, we have designed the task allocation and path planning as independent modules that can operate in parallel. Fig. 2 shows the flowchart of the MAPF and task allocation algorithms, organized as separate modules. The assignment of the next task group to a free agent is synchronized between the two modules, whereas all other processes operate in parallel. This synchronization is achieved by triggering the assignment process whenever a free agent request is received. The task allocation module consistently generates a task group for each agent and sequentially inserts it into a queue. This queue can hold up to N_{max_tg} task groups per agent, and when an agent becomes free, it is assigned the next task group. This approach ensures that the task allocation module has sufficient time to find high-quality solutions while the robots are in motion, without causing bottlenecks within the system. The proposed task allocation algorithm is described in Algorithm 1 and Section V.

Our framework is built on a decoupled architecture, where task assignment and path planning are managed as independent modules. Within this design, the critical responsibility for avoiding conflicts and ensuring safe navigation in the shared environment is exclusively handled by the path planning module. For this purpose, we employ the RHCR algorithm [11], a state-of-the-art method for lifelong MAPF. RHCR solves a series of windowed MAPF instances, focusing only on the near future rather than distant plans. It plans paths for all agents every λ timesteps, resolving collisions within a bounded horizon of ω timesteps. This method effectively minimizes computation time while still delivering high-quality results. Additionally, RHCR plans paths that traverse a sequence of goals rather than a single goal. If the distance to the last goal for an agent is less than ω, the agent is deemed free, and a new task group is assigned to it.

Algorithm 1 Task Group Allocation

Input: Set of agents $A = \{a_1, \ldots, a_N\}$, Set of initial tasks $T_{init} = \{\tau_1, \ldots, \tau_M\}$

```
T_list ← T_init
TG_list ← ∅
while True do
    A_free ← GetFreeAgents(A)
    if A_free ≠ ∅ and TG_list ≠ ∅ then
        /* Assign a task group for each free agent */
        AssignTaskGroups(A_free, TG_list)
    end if
    T_new ← GetNewTasks()
    if T_new ≠ ∅ then
        /* Update task groups from new tasks (Sect. IV.B) */
        T_unasgd ← UpdateTaskGroups(TG_list, T_new)
        T_list ← T_list ∪ T_unasgd
    else if T_list ≠ ∅ then
        /* Generate new task groups for agents (Sect. IV.A) */
        TG^l ← GenerateTaskGroups(A, T_list)
        TG_list ← TG_list ∪ TG^l
    end if
    UpdateAgentStates(A)
end while
```

V. TASK GROUP ALLOCATION

For each agent $a_i \in A$, the task allocation module continuously determines a task group TG_i^l from the pool of unassigned tasks T_{list} and inserts it into the task group set $TG_i = \{TG_i^1, \ldots, TG_i^L\}$. The task group sets for all agents are stored in the task group list $TG_{list} = \{TG_1, \ldots, TG_N\}$. Moreover, when a new task τ_{new} is released, the module evaluates whether to update an existing task group in TG_{list} by potentially replacing an assigned task with τ_{new}. The module then sequentially retrieves a task group from a task group set TG_i and assigns it to an agent a_i whenever a_i becomes free. This process ensures that the task allocation module continuously optimizes the composition of task groups and their execution sequence while agents carry out their tasks.

Let $V_{target} \subset V$ represent the set of target vertices for all tasks within a task group. In MT-MAPD, the goal is to arrange each task group to minimize the total travel distance across all vertices in V_{target}. To achieve this, we use the TSP algorithm [42] to compute the shortest path, p_{target}, which covers all vertices in V_{target}. Here, the path is calculated without considering collisions with other agents. For a given task group TG_i^l, we iteratively add a task that offers the shortest increase in p_{target} when incorporated into TG_i^l. This method arranges tasks by considering the closeness of their

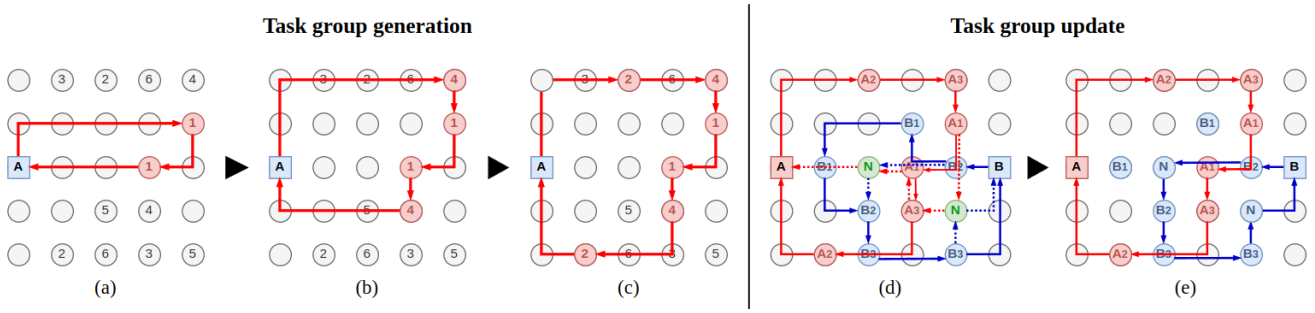

FIGURE 3. Illustration of the task group generation (a)–(c) and task group update (d)–(e) processes. Our method uses the TSP algorithm to calculate a tour that visits all targets in a task group and sequentially appends tasks with lower tour costs to the task group (a)–(c). When a new task (N) is released (d), the method evaluates whether replacing it with the currently allocated tasks (A1, A2, A3, B1, B2, and B3) reduces the task group's cost, and if so, performs the best replacement (e).

target vertices, enabling the creation of a task group that traverses all target vertices along the shortest possible route.

Algorithm 1 depicts the pseudocode of the proposed task group allocation algorithm. The algorithm initializes a task list T_{list} from the initial tasks $T_{init} = \{\tau_1, \ldots, \tau_M\}$ (Line 1) and a task group list TG_{list} (Line 2). Each task group $TG_i^l \in TG_{list}$ represents a set of tasks to be assigned to agent $a_i \in A$ for the l-th assignment. This algorithm iteratively generates a set of l-th task groups $TG^l = \{TG_1^l, \ldots, TG_N^l\}$ from continuously released tasks and sequentially assigns task groups to free agents (Lines 3 - 17).

For each iteration, the algorithm first checks for the presence of free agents A_{free} (Line 4). If there is a free agent a_i and its task group set $TG_i \subset TG_{list}$ is not empty (Line 5), the algorithm assigns the next task group TG_i^{front} at the front of TG_i to agent a_i and then removes TG_i^{front} from TG_{list}. Next, it checks for the presence of newly released tasks T_{new} (Line 8). If there is a released task $\tau_{new} \in T_{new}$, the algorithm evaluates whether replacing a task in any $TG_i^l \in TG_{list}$ with τ_{new} would lead to a higher quality solution and updates it accordingly (Line 10). At this stage, unassigned tasks T_{unasgd} are inserted into T_{list} to be utilized later for generating new task groups (Line 11). Next, the algorithm generates a set of task groups TG^l from the tasks in T_{list} (Line 13) and appends TG^l to the TG_{list} (Line 14). All tasks included in TG^l are removed from T_{list}.

Finally, at each iteration, the algorithm updates the existing TG_{list} with newly released tasks T_{new} or appends new task groups TG^l to TG_{list}. Consequently, by running these iterations in parallel while the agents are active, the task allocation module can efficiently determine the optimal sequence of task groups for each agent. Lines 4-6 can be implemented to trigger an immediate assignment whenever a free agent requests.

A. TASK GROUP GENERATION

This section describes the algorithm for generating a set of task groups, TG^l, from the tasks in T_{list}. The TG^l consists of a new task group TG_i^l for each agent $a_i \in A$. This algorithm computes the shortest path that visits all target vertices V_{target} of TG_i^l using the TSP algorithm and allocates a task based on the path cost. It determines the task that least increases the cost when added to TG_i^l and continues to add the best task τ_{best} to TG_i^l until the capacity of a_i allows.

Algorithm 2 Task Group Generation

Input: Set of agents $A = \{a_1, \ldots, a_N\}$, Set of initial tasks $T_{list} = \{\tau_1, \ldots, \tau_M\}$
Output: Set of new task groups $TG^l = \{TG_1^l, \ldots, TG_N^l\}$

1: $TG^l \leftarrow \emptyset$
2: $T_{sub} \leftarrow$ GetSubTasks(T_{list})
3: **for** each $a_i \in A$ **do**
4: $\quad TG_i^l \leftarrow \emptyset$
5: $\quad$ **while** $|TG_i^l| < a_i.capacity$ **and** IsUpdated(TG_i^l) **do**
6: $\quad\quad \tau_{best} \leftarrow \emptyset$
7: $\quad\quad TG_{best} \leftarrow \emptyset$
8: $\quad\quad$ **for** each $\tau_m \in T_{sub}$ **do**
9: $\quad\quad\quad$ **if** $|TG_i^l| + |\tau_m| > a_i.capacity$ **then**
10: $\quad\quad\quad\quad$ **continue**
11: $\quad\quad\quad$ **end if**
12: $\quad\quad\quad TG_{temp} \leftarrow$ MergeTask(τ_m, TG_i^l)
13: $\quad\quad\quad$ ComputeTSP(TG_{temp})
14: $\quad\quad\quad$ **if** $TG_{temp}.cost < TG_{best}.cost$ **then**
15: $\quad\quad\quad\quad \tau_{best} \leftarrow \tau_m$
16: $\quad\quad\quad\quad TG_{best} \leftarrow TG_{temp}$
17: $\quad\quad\quad$ **end if**
18: $\quad\quad$ **end for**
19: $\quad\quad TG_i^l \leftarrow TG_i^l \cup \tau_{best}$
20: $\quad\quad T_{sub} \leftarrow T_{sub} \setminus \tau_{best}$
21: $\quad$ **end while**
22: **end for**

Algorithm 2 depicts the pseudocode of the task group generation algorithm. For computational efficiency, the algorithm generates TG^l from a set of N_{sub} candidate tasks T_{sub} instead of all the tasks in T_{list} (Line 2). One of the objectives is to reduce service time, so tasks that are released early should be prioritized for assignment. We sort all tasks in T_{list} by release time and extract N_{sub} tasks for T_{sub}.

For each agent $a_i \in A$, the algorithm iteratively adds tasks one by one to TG_i^l until the capacity is full or no further updates occur (Lines 5-21). For each task $\tau_m \in T_{sub}$, the

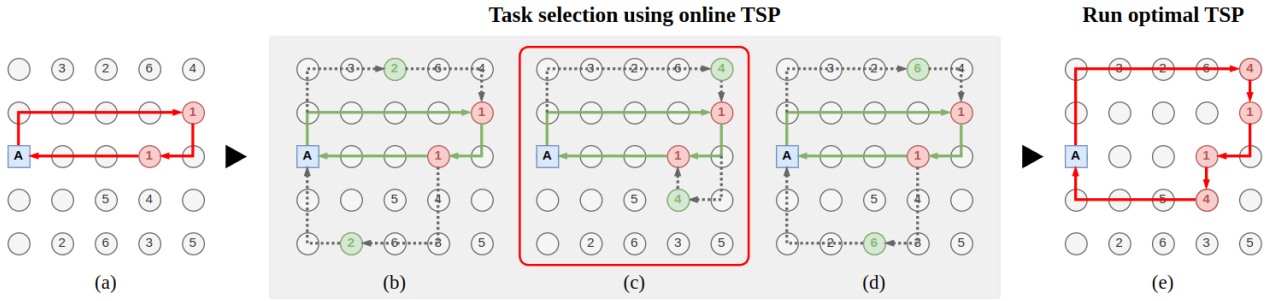

FIGURE 4. Illustration of the efficient task group generation process. Starting with an initial task group (a), the efficient method addresses the online TSP by reordering only the visitation sequence of the targets of a new task (2, 4, or 6) while maintaining the order of the existing targets. The online TSP determines the shortest detour sequence for the new targets while preserving the visitation order of the existing ones (b) - (d). Finally, the method incorporates the task with the lowest detour cost into the task group (c) and recalculates the visitation sequence using the original TSP algorithm [42] (e).

algorithm creates a temporal task group TG_{temp} by adding τ_m to TG_i (Line 12). It then computes a path that visits all target vertices of TG_{temp} using the TSP algorithm (Line 13). If the cost of this path is the lowest so far, τ_m is recorded as the best task, τ_{best}. Finally, the selected task, τ_{best}, is assigned to TG_i^l (Line 19), and τ_{best} is removed from T_{sub} (Line 20). After performing the above process for all agents, it is possible to generate a TG_i^l for each agent that has the shortest travel path. Figs. 3a–3c provide an example illustrating the proposed task group generation process.

B. UPDATING TASK GROUPS WITH NEW TASKS

Our algorithm continuously updates the existing task groups in TG_{list} with new tasks T_{new} as they are released, as outlined in Line 10 of Algorithm 1. We update a task group $TG_i^l \in TG_{list}$ by replacing an existing task $\tau_m \in TG_i^l$ with a new task $\tau_{new} \in T_{new}$ if this exchange leads to an improved solution. Let T_{asgd} be the set of all tasks currently assigned to any task group in TG_{list}. To enhance efficiency, we evaluate only a subset T_{asgd_sub} of T_{asgd}, rather than the entire set. This subset is chosen by selecting up to N_{asgd_sub} tasks that were released t_{update} timesteps before the current timestep from T_{asgd}.

For each task $\tau_m \in T_{asgd\_sub}$, we tentatively replace it with a new task $\tau_{new} \in T_{new}$ and recalculate the cost of visiting all target vertices using the TSP algorithm. We then identify the best task $\tau_{best} \in T_{sub}$ that achieves the largest reduction in cost and proceed with the replacement of τ_{best} with τ_{new}. The task τ_{best} that was replaced is then moved to the set of unassigned tasks T_{unasgd}. If τ_{new} does not lead to any cost reduction, it is also added to T_{unasgd} without any update. Finally, the tasks that are unassigned or have been replaced in T_{unasgd} are used to generate new task groups in subsequent operations. This approach ensures that task groups are dynamically adjusted to incorporate new, potentially more efficient tasks, thereby optimizing the overall solution continuously. Figs. 3d and 3e show an example of the task group update process.

C. EFFICIENCY IMPROVEMENT

The task group generator (Algorithm 2) needs to execute the TSP algorithm up to N_{sub} times for all tasks in T_{sub} when selecting a single task $\tau_m \in T_{sub}$ for a task group TG_i^l. This approach also involves repeatedly running multiple TSP computations until the capacity is reached. Such a process demands extensive computation and increasingly takes longer to form all task groups in TG^l as the number of agents grows. While the proposed method ensures there is sufficient time to compute task groups during robot movement, it can result in bottlenecks if the computation time grows significantly.

To address this, we additionally introduce an efficient task group generator that substantially lowers the computational burden of the TSP. Given the existing task group's targets V_{target} and the new task's targets V_{target}^{new}, the original algorithm (Algorithm 2) computes the TSP on the combined set of all targets. In contrast, the efficient version solves an online TSP by only reordering the visitation sequence of the new targets in V_{target}^{new}, while preserving the sequence of the existing targets in V_{target}. For each new target $v^{new} \in V_{target}^{new}$, the method identifies a detour path by inserting it between two consecutive vertices in the current sequence.

Let $\{V_{\pi_1}, \ldots, V_{\pi_K}\}$ be the current sequence of V_{target}, where $\Pi = \{\pi_1, \ldots, \pi_K\}$ is the permutation of the target indices $\{1, \ldots, K\}$, representing the order of visitation. For each pair of consecutive targets $(v_{\pi_k}, v_{\pi_{k+1}})$, we compute the detour path distance for a new target $v^{new} \in V_{target}^{new}$ as follows:

$$D(v_{\pi_k}, v_{\pi_{k+1}}, v^{new}) = dist(v_{\pi_k}, v^{new}) + dist(v^{new}, v_{\pi_{k+1}})$$

where $dist(v_A, v_B)$ is the distance of the path starting at v_A and ending at v_B on the graph G. This distance can be quickly estimated using precomputed heuristic values utilized by the MAPF solver [11]. Next, we identify the consecutive target pair $(v_{\pi_k}, v_{\pi_{k+1}})$ where $D(\cdot, \cdot, \cdot)$ is minimized. We then update the sequence of V_{target} by inserting v^{new} between v_{π_k} and $v_{\pi_{k+1}}$. By repeating this process for all new targets, we determine the shortest detour path that incorporates the new targets into the existing sequence.

By applying this method to Line 13 of Algorithm 2, we can quickly identify the best task τ_{best} without the need for extensive TSP computations. Additionally, after Line 20 of Algorithm 2, we perform the TSP just once on the updated targets of TG_i^l. This allows us to define the optimal visiting sequence for the targets of TG_i^l, which can then be utilized when selecting a task in subsequent iteration. Fig. 4 shows an

illustration of the processing steps in the proposed efficient method for task group generation.

Our online TSP updater employs the cheapest-insertion heuristic [43], [44], which iteratively adds targets to the tour at positions that minimally increase its length. This method provides a 2-approximation guarantee, ensuring the tour is at most twice the cost of the optimal one. In our system, this insertion rule is applied during task-group generation to evaluate and select new tasks. Once a group's targets are finalized, we run the original TSP solver [42] once to recompute the global visitation sequence, thereby mitigating the suboptimality of purely sequential insertions. Empirical validation in Section VI-E shows that the average cost of our online tours is only about 1.09 times that of the original TSP solutions across all scenarios.

Complexity: We adopted the 2-opt algorithm for TSP computation [42], which has a complexity of $O(K^2)$, where K is the number of targets in a task group. Consequently, the complexity of the original task group generation algorithm (Algorithm 2) becomes $O(N \cdot C \cdot M \cdot K^2)$, where N is the number of agents, C is the capacity, and M is the number of tasks. On the other hand, the complexity of the efficient algorithm proposed in this section is $O(N \cdot C \cdot K^2)$. Unlike the original algorithm, the efficient algorithm computes TSP only for the targets in the final selected TG_{best}, rather than for each task. Therefore, while the original algorithm may perform TSP computations up to $N \cdot C \cdot M$ times, the efficient algorithm reduces this to a maximum of $N \cdot C$ computations, significantly decreasing computation time. The performance of the proposed efficient method is discussed in detail in Section VI-E.

VI. EXPERIMENTAL RESULTS

We considered the proposed method in two ways: an efficient version that applies the approach from Section V-C (**Ours$_{\text{eff}}$**) and an optimal version that does not (**Ours$_{\text{opt}}$**). We compared the performance of the proposed methods with the following MT-MAPD methods:

- **TP** [6]: This method forms a task group with tasks that have the shortest average distance between the agent's current location and multiple targets, instead of one target.
- **TSP-MAPD** [17]: This method randomly assigns a released task to a free agent, ensuring that its capacity is not exceeded. Once a task is assigned, the visiting sequence of targets is determined using the TSP algorithm.
- **TP-TSP**: This is an integrated method combining TP and TSP-MAPD. It generates a task group using TP and then determines the visiting sequence of targets using the TSP algorithm.

All methods were implemented in C++ and executed on a standard desktop PC equipped with an Intel Core i7-6700K CPU and 64 GB of RAM, without using a graphics processing unit. We set N_{sub} and N_{asgd_sub} to $1.5 \times N(\text{number of agents}) \times N_{max}$, where N_{max} is the maximum number of tasks that can be included in a task group with a capacity. Similar to our method, TP and TP-TSP also selected N_{sub} tasks instead of the entire set of tasks and generated task groups from them. We set N_{max_tg} to 10 and t_{update} to 30 timesteps. For MAPF, we set the parameters of RHCR [11] to $\lambda = 5$ and $\omega = 10$. Thus, the path planning module replans the multi-agent paths every 5 timesteps, while the task allocation module also generates or updates new task groups in parallel every 5 timesteps. We used ECBS [21] as the MAPF solver and set its suboptimality factor to 1.5.

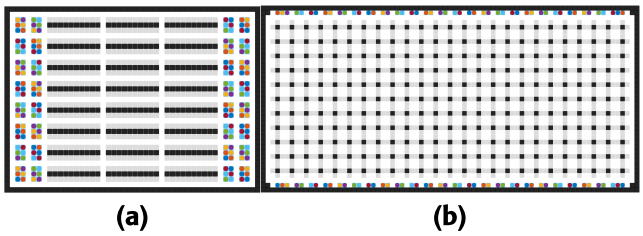

FIGURE 5. Simulated (a) Kiva warehouse (48 × 36) and (b) sorting center (77 × 37) environments used in the experiments. Black areas indicate static obstacles, gray areas are pickup points (target locations), and other color points denote home locations.

For our experiments, we used the LKH TSP solver with identical parameter settings for both the offline and online phases across all compared methods to ensure a fair and consistent evaluation. LKH adaptively selects the k in k-opt and is widely regarded as a state-of-the-art heuristic for producing high-quality approximate solutions to the TSP. In our experiments, we initialized LKH to allow up to 5-opt moves (adaptive k), with an alpha-nearness candidate set, 20 candidate edges per node, and a patching-cycle limit of 3.

We employed two benchmark maps [11] for our evaluation: the Kiva warehouse (Fig. 5a) and the sorting center (Fig. 5b), both in their undirected versions. For each scenario, we randomly generated 2600 tasks, each consisting of three targets. Of those tasks, 600 are released initially, and the remaining tasks are released in batches of 10 every 5 timesteps, up to 1000 timesteps. To ensure a fair evaluation, all methods were tested on the same task sets. To process a task group, an agent visits each target in sequence and then moves to the drop-off location, which is the agent's home location. If an agent completes all its tasks and no further task groups remain, it returns home location and waits until the scenario is complete. We evaluated each method using two metrics: makespan and average service time. The makespan measures the total time to complete all released tasks, while the average service time is the average delay between a task's release and its completion.

A. EVALUATION OF MT-MAPD

Tables 2 and 3 show the performance of each method, measured by average service time, makespan, and runtime per timestep, for different numbers of agents (N) and capacities

TABLE 2. Performance evaluation results of MT-MAPD in the Kiva warehouse scenario.

Kiva		TP			TSP-MAPD			TP-TSP			Ours_{eff}			Ours_{opt}		
N	C	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)
40	6	2451.69	6531	0.28	1760.79	4456	1.11	1468.63	3990	1.22	1321.42	3652	55.12	**1283.64**	**3560**	152.74
60	6	1640.28	4679	0.60	1123.31	3176	1.67	924.86	2805	1.91	805.82	2553	67.45	**778.53**	**2530**	258.72
80	6	1297.45	3997	1.11	818.86	2511	2.35	675.59	2249	2.70	567.46	2011	73.06	**548.25**	**1957**	413.94
100	6	1145.12	3722	1.92	647.75	2111	3.21	557.32	2070	3.60	441.26	1783	74.83	**430.28**	**1756**	546.41
40	9	2369.19	6191	0.31	1308.93	3486	1.18	1071.67	3037	1.88	889.28	2656	83.57	**841.13**	**2558**	312.52
60	9	1627.98	4647	0.65	810.22	2465	2.18	651.18	2118	2.17	496.30	1838	86.53	**468.38**	**1775**	582.58
80	9	1320.33	3922	1.26	573.43	1918	2.49	452.31	1686	3.02	320.33	1475	91.03	**296.59**	**1425**	891.46
100	9	1196.04	3827	2.06	438.60	1666	3.26	352.15	1509	4.35	243.58	1301	95.84	**231.84**	**1262**	1157.16

TABLE 3. Performance evaluation results of MT-MAPD in the sorting center scenario.

Sorting		TP			TSP-MAPD			TP-TSP			Ours_{eff}			Ours_{opt}		
N	C	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)	service time	makespan	runtime /step(ms)
40	6	3463.03	8962	0.23	2562.92	6161	0.84	2082.32	5335	0.94	1931.52	4980	41.19	**1882.83**	**4868**	113.82
60	6	2211.33	5957	0.38	1626.73	4262	1.26	1276.53	3528	1.47	1167.18	3350	53.23	**1131.29**	**3253**	212.81
80	6	1588.04	4627	0.55	1152.68	3193	1.74	876.34	2725	1.96	788.34	2514	62.10	**762.38**	**2472**	332.12
100	6	1214.11	3641	0.75	870.45	2581	2.23	636.63	2190	2.49	564.92	2017	66.04	**539.79**	**1942**	499.62
40	9	3339.19	8547	0.23	1875.39	4671	0.96	1489.92	3963	1.12	1299.50	3517	69.71	**1247.04**	**3411**	247.47
60	9	2147.61	5630	0.39	1165.31	3216	1.42	886.36	2684	1.70	747.20	2399	80.94	**711.16**	**2282**	476.93
80	9	1550.48	4312	0.56	808.74	2478	1.91	587.00	2073	2.29	474.87	1800	82.82	**445.05**	**1745**	764.21
100	9	1201.50	3594	2.09	596.19	2035	2.71	406.31	1689	2.87	318.93	1563	96.02	**293.07**	**1517**	1015.55

(C). The performance represents the average results of 5 experiments. TP had the worst performance in terms of average service time and makespan because it does not determine the optimal visiting sequence for the targets. TP-TSP performed much better in terms of average service time and makespan than TSP-MAPD, while their runtimes were similar. This indicates that forming task groups with tasks close to the agents' locations is much more effective than selecting tasks randomly. Notably, as N increases, the effectiveness of TP-TSP significantly improves compared to TSP-MAPD.

Ours_{opt} generally exhibited the best average service time and makespan performance in all scenarios, and especially performed better in environments with large number of agents (N) and capacity (C). In both scenarios, when $N = 100$ and $C = 9$, Ours_{opt} reduced the average service time by 31.1% and the makespan by 13.4% on average compared to TP-TSP. In particular, compared to the worst method, TP, with the same case, Ours_{opt} reduced the average service time by 362.9%(4.6*times*) and the makespan by 170%(2.7*times*). Ours_{opt} generates task groups by considering the optimal route for visiting targets, rather than just a single location. Therefore, it can produce task groups of much higher quality than the other methods.

However, Ours_{opt} showed the worst performance in terms of runtime. Especially when $N = 100$, Ours_{opt} requires an enormous amount of computation time, making online computation impossible. On the other hand, Ours_{eff} was able to significantly reduce runtime while maintaining a high-quality solution. The runtime of Ours_{eff} is, on average, approximately 7.7 times faster than Ours_{opt}. Notably, in the most computationally intensive case ($N = 100$ and $C = 9$), the runtime of Ours_{eff} is 11.3 times faster than Ours_{opt}. Although Ours_{eff} has slightly higher average service time and makespan than Ours_{opt}, the difference is not significant when considering the reduction in runtime. Ours_{eff} can perform online computations while simultaneously producing high-quality solutions. This indicates that the efficiency improvement method proposed in Section V-C is highly effective.

We also visualized the movement paths of agents performing a task group for each method. Fig. 6 illustrates the paths of representative agents visiting each target in both scenarios ($N = 100$ and $C = 9$). As shown in Fig. 6, TSP-MAPD and TP-TSP generated overlapping, lengthy, and inefficient paths. Specifically, TP-TSP produces circuitous paths when tasks are not appropriately allocated, even though it tries to assign the closest tasks. In contrast, our methods (Ours_{opt}

FIGURE 6. Illustration of the movement paths of representative agents (8 agents on the Kiva map (a)–(d) and 10 agents on the sorting center map (e)–(h)) executing task groups generated by each method with $N = 100$ and $C = 9$. The average cost refers to the average travel distance of the paths visiting the targets in each task group.

and Ours$_{eff}$) generated shorter paths with significantly less overlap compared to other methods. This demonstrates the effectiveness of determining the optimal visit sequence when forming task groups.

Our method outperforms prior approaches because it optimizes the tour for an entire task group rather than selecting tasks based on individual proximity. Proximity-based policies, such as TP and TP-TSP, sequentially select the task nearest to the agent's current pose. While this minimizes immediate travel distance, it can result in globally inefficient paths if the selected tasks are spatially dispersed. In contrast, our approach leverages a TSP algorithm to evaluate the total tour cost of a candidate task group and uses this metric as the primary selection criterion. This allows the method to form groups of tasks that, while potentially distant from each other, collectively constitute a highly efficient route. As visually demonstrated in Fig. 6, our strategy generates significantly shorter and more consolidated paths with less overlap.

B. EFFECTIVENESS OF TASK GROUP UPDATE

We conducted an ablation study to verify the effectiveness of the update step t_{update} described in Section V-B. In the task group update step, newly released tasks are compared and potentially replaced with tasks that were released up to t_{update} timesteps prior. As t_{update} increases, our methods update the task groups by comparing more tasks, thereby applying the more effects of the update step. We measured performance by varying t_{update} from 0 to 150 in the Kiva scenario with the same parameter of Section VI-A. To evaluate the performance of task group updates for a larger and more diverse set of tasks, we released 6, 500 random tasks over 3, 000 timesteps and assessed the performance.

Fig. 7 shows the average service time and runtime per timestep of our methods as t_{update} changes. Both Ours$_{opt}$ and Ours$_{eff}$ exhibit better average service time performance as t_{update} increases. In particular, Ours$_{eff}$ showed the most significant performance improvement at $t_{update} = 60$.

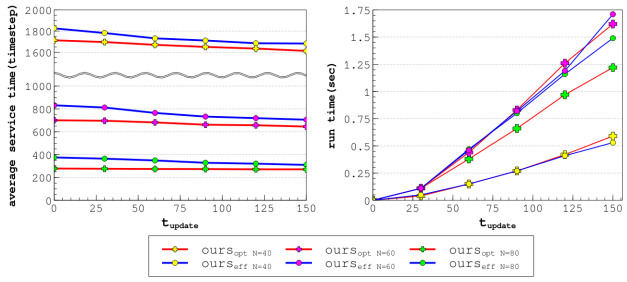

FIGURE 7. Evaluation of the effect of the task group update step. In the Kiva scenario ($C = 9$), changes in t_{update} were measured for (*left*) average service time and (*right*) runtime per timestep.

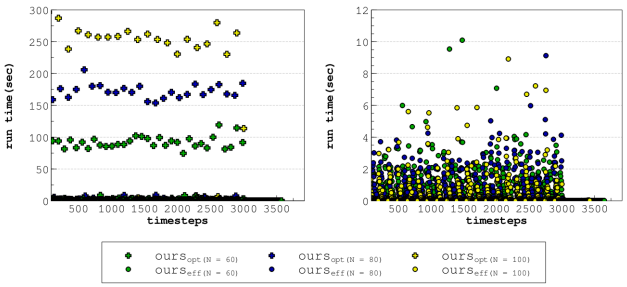

FIGURE 8. Runtime of (*left*) Ours_{opt} and (*right*) Ours_{eff} at each timestep for the Kiva scenario ($C = 9$).

However, when t_{update} is large, the number of comparing operations for update increases, which increases computation time significantly. In particular, the runtime increased greatly when t_{update} reached 120. Therefore, an appropriate t_{update} should be determined according to a time budget. These results indicate that the update step with an appropriate t_{update} can considerably contribute to the performance enhancement.

In summary, the continuous arrival of new tasks in online scenarios can render previously assigned task groups suboptimal. To address this, our method incorporates a dynamic task group update mechanism that replaces an existing task with a new arrival if doing so reduces the total tour cost. This continuous update is central to our strategy's adaptability and sustained efficiency. A key consideration, however, is the trade-off between solution quality and computational load. Increasing the t_{update} parameter broadens the search scope for potential replacements, thereby improving solution quality through evaluation of more candidates, but at the expense of higher computation time. Therefore, in practical applications, this parameter must be carefully calibrated to balance performance and computational constraints.

C. COMPUTATION TIME ANALYSIS

In this experiment, we conduct an in-depth analysis of the computation time of the proposed methods. Our method maintains only $N_{max\_tg} = 10$ task groups in a task group list TG_i for each agent a_i. Therefore, our methods do not generate task groups for all agents in every iteration; in most cases, only several task groups are generated. Consequently, we measured the runtime at each timestep to analyze the variation in runtime.

Fig. 8 shows the changes in runtime at each timestep for the Kiva scenario with Ours_{opt} and Ours_{eff}. Ours_{opt} typically has a runtime of less than 5 seconds at most timesteps. However, Ours_{opt} requires significantly more computational time at several timesteps, particularly when $N = 100$, with 20 instances taking more than 200 seconds each. Our method, while capable of computing task assignments in parallel as the robots move, may still experience bottlenecks if there is no task group available to assign in a task group list.

In contrast, Ours_{eff} can perform computations in less than 3 seconds at most timesteps. In this experiment, the fastest time observed for an agent to complete a task group was 45 timesteps. Roughly assuming that 1 timestep takes 1 second in real time (considering that actual Kiva robots move at approximately 1.3 to 1.5 meters per second, and the distance per grid node is about 1.5 to 2 meters), it can be estimated that a robot would take at least 40 seconds to complete a task group. As shown in Fig. 8, Ours_{eff} can produce task groups within 10 seconds each time, which is much shorter than the minimum completion time of 40 seconds. This means that Ours_{eff}, when processing path planning and task allocation in parallel in a real-world scenario, does not encounter bottlenecks.

Consistent with the complexity analysis in Section V-C, the computation time for both our optimal (Ours_{opt}) and efficient (Ours_{eff}) methods scales linearly with the number of agents, N. Consequently, for large-scale fleets (e.g., a thousand agents), the low computational overhead of Ours_{eff} ensures robust online performance. In contrast, the substantial computation required by Ours_{opt} would render it infeasible for real-time applications at such a scale. The computation time also scales linearly with the number of available tasks, M. However, our approach is designed to manage a large task pool effectively by adjusting the N_{sub} parameter. This mechanism allows us to control the number of candidate tasks evaluated during each task group generation step, a topic that is further detailed in the subsequent subsection.

D. INFLUENCE OF SUBTASK SIZE

In this section, we examine the impact of the subtask size N_{sub} on the performance of task group generation. For computational efficiency, our algorithm (Algorithm 2) generates a task group from a subset T_{sub} containing N_{sub} candidate tasks, rather than utilizing all tasks in T_{list}. We evaluated the runtime and average service time of the generated task groups by varying T_{sub} from 240 to 420, using 80 agents.

Table 4 presents the performance of Ours_{opt} and Ours_{eff} as N_{sub} changes. As N_{sub} increases, both methods generate better task groups with a shorter service time due to the availability of a larger pool of candidates. In terms of runtime, Ours_{opt} requires significantly more runtime proportional to the subtask size. In contrast, Ours_{eff} showed minimal

TABLE 4. Average service time and runtime of Ours_{opt} and Ours_{eff} ($N = 80$ and $C = 9$) based on subtask size N_{sub}.

SCENARIO	N_{sub}	Ours_{opt} service time	Ours_{opt} runtime /step(ms)	Ours_{eff} service time	Ours_{eff} runtime /step(ms)
Kiva map	240	299.66	804.05	321.14	89.52
	300	302.77	843.97	320.84	90.94
	360	298.70	905.45	320.60	89.77
	420	290.74	1520.19	313.49	82.23
Sorting map	240	456.92	663.28	488.89	83.13
	300	452.28	734.28	480.52	84.12
	360	447.65	766.30	475.34	82.76
	420	421.29	1295.49	447.95	72.21

TABLE 5. Performance comparison between the original TSP (TSP_{opt}) and the online TSP (TSP_{eff}). The evaluation of both methods was conducted by calculating the tour visiting the targets as four tasks were sequentially added, in the same order, to a task group.

SCENARIO	C	run time TSP_{opt} (ms)	run time TSP_{eff} (ms)	cost TSP_{opt}	cost TSP_{eff}	cost eff/opt
Kiva map	3	2.6250	0.0009	93.58	96.90	1.04
	6	2.9895	0.0017	119.92	141.30	1.18
	9	3.7170	0.0022	140.90	157.14	1.12
	12	4.5652	0.0028	159.34	171.94	1.08
Sorting map	3	2.638	0.001	134.56	139.24	1.04
	6	2.9898	0.0019	171.15	200.76	1.17
	9	3.9347	0.0026	194.90	218.93	1.12
	12	4.9044	0.0033	214.25	235.94	1.10

variation in runtime and even achieved its best runtime performance when $N_{sub} = 420$. With a larger subtask size, Ours_{eff} generated the most optimal solutions, leading to significantly fewer task group updates. This reduction in updates decreased computational overhead, ultimately resulting in the best runtime performance.

Although Ours_{eff} generates task groups with an average service time of approximately 6.8% higher than Ours_{opt}, the runtime efficiency of Ours_{eff} makes this performance gap insignificant. Furthermore, in the sorting map scenario, at $N_{sub} = 420$, Ours_{eff} outperforms Ours_{opt} at $N_{sub} = 300$ while achieving significant runtime savings. This shows that Ours_{eff} can handle larger subtask sizes efficiently, whereas Ours_{opt} faces challenges with high N_{sub}.

E. EFFECTIVENESS OF ONLINE TSP

We evaluate the performance of the online TSP algorithm proposed in Section V-C. The online TSP algorithm continuously updates the visitation sequence of targets according to incoming targets. It computes detour paths for newly added targets while preserving the previously calculated visitation sequence. This approach enables much faster computation by determining the visitation sequence for newly added targets only, instead of recalculating the entire sequence for all targets.

We compared the performance of the online TSP algorithm with the original TSP algorithm [42]. We consider a task group consisting of 4 tasks (12 targets) and assume that the input order of the tasks is predetermined. For each task group, we sequentially added tasks one by one and calculated the visitation route for the input targets using each TSP algorithm.

Table 5 presents the average cost (path distance) and runtime at each step for 1,000 task groups. As shown in the table, the runtime of the online TSP method is, on average, approximately 1,900 times faster than the original TSP method, while maintaining solution costs at approximately 1.11 times those of the original. In the last step, the online TSP achieved solutions with an average cost within 1.09 times the original TSP, demonstrating that the proposed method can deliver sufficiently high-quality solutions. This also indicates that the proposed TSP method becomes more effective as the number of targets increases.

VII. LIMITATIONS AND DISCUSSION

Our method offers several key advantages for online multi-agent pickup and delivery scenarios. First, by optimizing task-group sequences within a global routing context, it significantly reduces path overlap and alleviates bottleneck congestion. Second, our efficient variant (Ours_{eff}) achieves a favorable balance between solution quality and computational latency; it operates up to 11.3× faster than the optimal version with only marginal performance degradation, rendering it highly suitable for real-time applications. Finally, the computational latency per update remains well below typical task execution times. This ensures robust service levels and high throughput, even under dynamic conditions with high task arrival rates.

While the proposed approach delivers promising results in experiments, it still has several limitations. First, the performance of the proposed method can be significantly influenced by the order in which task groups are generated. The algorithm generates task groups for agents sequentially, starting from a_1 to a_N (Line 3 of Algorithm 2). However, this sequential approach may fail to find the best solution if tasks assigned to an earlier task group could have resulted in a lower cost for a later task group. A potential solution to this issue would involve comparing task groups across all agents for each task; however, this approach would require N-times more computation, leading to a significant increase in runtime. As a practical compromise, this study adopts the sequential allocation method. Future research could investigate methods that compare subsets of task groups instead of all task groups, potentially achieving a better balance between solution quality and computational efficiency.

Second, the task group update phase requires significant computation, making it essential to set an appropriate t_{update} parameter. During this phase, the algorithm checks whether replacing already allocated tasks with new ones improves the cost of a task group, and if so, performs the replacement. When evaluating task replacement, the visitation sequence must be recalculated using the original TSP algorithm. At this phase, the efficient method of the online TSP algorithm cannot be applied, making it difficult to evaluate replacements involving a large number of tasks. Therefore, it is crucial to set an appropriate t_{update} value, considering the number of agents and the time available for computation.

Third, while this paper establishes a robust framework, its current scope does not address precedence constraints among tasks or the complexities of heterogeneous robot-task compatibility. Future work will extend this framework to incorporate these practical considerations. Precedence constraints can be integrated by modifying the TSP tour cost, wherein any violation is treated as either a hard constraint (rendering the sequence infeasible) or a soft constraint (imposing a significant penalty). This ensures the optimization naturally yields a valid visitation sequence. Similarly, heterogeneous agent-task assignments can be managed by first pre-filtering candidate tasks for compatibility and then integrating a robot-task suitability score into the cost function for task group generation.

Finally, the proposed method uses a decoupled approach, handling task allocation and path planning as separate processes. This means task allocation focuses solely on the travel distance between targets, without accounting for potential collisions in the paths. In the Kiva map and Sorting Center map used in this study, the decoupled approach was suitable because alternative routes to avoid collisions between agents were easily found. However, in environments with narrow corridors [38], [45], deadlocks may occur more frequently, and agents may be forced to take significantly longer detour routes. In such scenarios, incorporating path planning into the task allocation process becomes essential. Developing a coupled approach that simultaneously considers task allocation and path planning could be a promising direction for future work.

VIII. CONCLUSION

This study addressed the MAPD problem, focusing on the multi-task MAPD variant, where tasks consist of multiple targets, and agents are assigned several tasks to complete in a single trip. We proposed a novel task allocation algorithm that leverages the TSP to determine optimal visitation sequences, ensuring efficient task group generation for online task releases. By iteratively identifying tasks with the shortest detour paths, the algorithm guarantees that task groups have the shortest possible travel paths. To enhance computational efficiency, we introduced an online TSP algorithm that recalculates visitation sequences only for newly added targets, significantly accelerating the computation time compared to recalculating for all targets. Extensive experiments on benchmark scenarios demonstrated that our approach outperforms existing state-of-the-art methods, achieving a notable reduction in service time.

REFERENCES

[1] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," *AI Mag.*, vol. 29, no. 1, p. 9, 2007.

[2] A. Bolu and Ö. Korçak, "Adaptive task planning for multi-robot smart warehouse," *IEEE Access*, vol. 9, pp. 27346–27358, 2021.

[3] D. Shi, Y. Tong, Z. Zhou, K. Xu, W. Tan, and H. Li, "Adaptive task planning for large-scale robotized warehouses," in *Proc. IEEE 38th Int. Conf. Data Eng. (ICDE)*, May 2022, pp. 3327–3339.

[4] T. M. Ho, K.-K. Nguyen, and M. Cheriet, "Federated deep reinforcement learning for task scheduling in heterogeneous autonomous robotic system," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 528–540, Jan. 2024.

[5] Y. Lin, Y. Xu, J. Zhu, X. Wang, L. Wang, and G. Hu, "MLATSO: A method for task scheduling optimization in multi-load AGVs-based systems," *Robot. Comput.-Integr. Manuf.*, vol. 79, Feb. 2023, Art. no. 102397.

[6] H. Ma, J. Li, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding for online pickup and delivery tasks," in *Proc. 16th Conf. Auto. Agents MultiAgent Syst.*, 2017, pp. 837–845.

[7] M. Liu, H. Ma, J. Li, and S. Koenig, "Task and path planning for multi-agent pickup and delivery," in *Proc. 18th Int. Conf. Auto. Agents MultiAgent Syst.*, 2019, pp. 1152–1160.

[8] H. Ma, W. Hönig, T. K. S. Kumar, N. Ayanian, and S. Koenig, "Lifelong path planning with kinematic constraints for multi-agent pickup and delivery," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 1, pp. 7651–7658.

[9] A. Fenoy, J. Zagoli, F. Bistaffa, and A. Farinelli, "Attention for the allocation of tasks in multi-agent pickup and delivery," in *Proc. 39th ACM/SIGAPP Symp. Appl. Comput.*, Apr. 2024, pp. 622–629.

[10] F. Grenouilleau, W. V. Hoeve, and J. Hooker, "A multi-label A algorithm for multi-agent pathfinding," in *Proc. Int. Conf. Automated Planning Scheduling*, vol. 29, 2019, pp. 181–185.

[11] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding in large-scale warehouses," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 13, pp. 11272–11281.

[12] Q. Xu, J. Li, S. Koenig, and H. Ma, "Multi-goal multi-agent pickup and delivery," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 9964–9971.

[13] Z. Chen, J. Alonso-Mora, X. Bai, D. D. Harabor, and P. J. Stuckey, "Integrated task assignment and path planning for capacitated multi-agent pickup and delivery," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5816–5823, Jul. 2021.

[14] Y. Li, R. Huang, H. Ye, H. Huang, and H. Du, "Dynamic path finding for multi-load agent pickup and delivery problem," *Theor. Comput. Sci.*, vol. 1023, Jan. 2025, Art. no. 114897.

[15] H. A. Aryadi, R. Bezerra, K. Ohno, K. Gunji, S. Kojima, M. Kuwahara, Y. Okada, M. Konyo, and S. Tadokoro, "Multi-agent pickup and delivery in transformable production," in *Proc. IEEE 19th Int. Conf. Autom. Sci. Eng. (CASE)*, Aug. 2023, pp. 1–8.

[16] S. Teck, P. Vansteenwegen, and R. Dewil, "An efficient multi-agent approach to order picking and robot scheduling in a robotic mobile fulfillment system," *Simul. Model. Pract. Theory*, vol. 127, Sep. 2023, Art. no. 102789.

[17] F. Kudo and K. Cai, "A TSP-based online algorithm for multi-task multi-agent pickup and delivery," *IEEE Robot. Autom. Lett.*, vol. 8, no. 9, pp. 5910–5917, Sep. 2023.

[18] A. Khamis, A. Hussein, and A. Elmogy, "Multi-robot task allocation: A review of the state-of-the-art," in *Cooperative Robots and Sensor Networks 2015*. Cham, Switzerland: Springer, 2015, pp. 31–51.

[19] X. Bai, A. Fielbaum, M. Kronmüller, L. Knoedler, and J. Alonso-Mora, "Group-based distributed auction algorithms for multi-robot task assignment," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1292–1303, Apr. 2023.

[20] Z. Liu, H. Wei, H. Wang, H. Li, and H. Wang, "Integrated task allocation and path coordination for large-scale robot networks with uncertainties," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 4, pp. 2750–2761, Oct. 2022.

[21] M. Barer, G. Sharon, R. Stern, and A. Felner, "Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem," in *Proc. Int. Symp. Combinat. Search*, 2021, vol. 5, no. 1, pp. 19–27.
[22] E. Boyarski, A. Felner, R. Stern, G. Sharon, O. Betzalel, D. Tolpin, and E. Shimony, "ICBS: The improved conflict-based search algorithm for multi-agent pathfinding," in *Proc. Int. Symp. Combinat. Search*, 2021, vol. 6, no. 1, pp. 223–225.
[23] M. Damani, Z. Luo, E. Wenzel, and G. Sartoretti, "PRIMAL$_2$: Pathfinding via reinforcement and imitation multi-agent learning–lifelong," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 2666–2673, Apr. 2021.
[24] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig, "Searching with consistent prioritization for multi-agent path finding," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 188–189.
[25] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. K. S. Kumar, S. Koenig, and H. Choset, "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2378–2385, Jul. 2019.
[26] H. Ma and S. Koenig, "Optimal target assignment and path finding for teams of agents," in *Proc. Int. Conf. Auto. Agents Multiagent Syst.*, 2016, pp. 1144–1152.
[27] W. Hönig, S. Kiesel, A. Tinka, J. W. Durham, and N. Ayanian, "Conflict-based search with optimal task assignment," in *Proc. Int. Joint Conf. Auto. Agents Multiagent Syst.*, 2018, pp. 757–765.
[28] Z. Ren, S. Rathinam, and H. Choset, "Conflict-based Steiner search for multi-agent combinatorial path finding," in *Proc. Robot., Sci. Syst.*, Jun. 2022, pp. 1–13.
[29] X. Zhong, J. Li, S. Koenig, and H. Ma, "Optimal and bounded-suboptimal multi-goal task assignment and path finding," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 10731–10737.
[30] P. Surynek, "Multi-goal multi-agent path finding via decoupled and integrated goal vertex ordering," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 12, no. 1, pp. 197–199.
[31] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artif. Intell.*, vol. 219, pp. 40–66, Feb. 2015.
[32] F. Ho, R. Higa, T. Kato, and S. Nakadai, "A hierarchical approach for joint task allocation and path planning," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 10137–10144, Nov. 2024.
[33] Z. Ren, S. Rathinam, and H. Choset, "CBSS: A new approach for multiagent combinatorial path finding," *IEEE Trans. Robot.*, vol. 39, no. 4, pp. 2669–2683, Aug. 2023.
[34] Z. Ren, S. Rathinam, and H. Choset, "A conflict-based search framework for multiobjective multiagent path finding," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1262–1274, Apr. 2023.
[35] Y. Li and H. Huang, "Efficient task planning for heterogeneous AGVs in warehouses," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 10005–10019, Aug. 2024.
[36] X. Bai, W. Yan, and S. S. Ge, "Distributed task assignment for multiple robots under limited communication range," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 7, pp. 4259–4271, Jul. 2022.
[37] F. Kudo and K. Cai, "Anytime multi-task multi-agent pickup and delivery under energy constraint," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 10145–10152, Nov. 2024.
[38] S. Song, K.-I. Na, and W. Yu, "Anytime lifelong multi-agent pathfinding in topological maps," *IEEE Access*, vol. 11, pp. 20365–20380, 2023.
[39] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transp. Sci.*, vol. 40, no. 4, pp. 455–472, Nov. 2006.
[40] X. Bai, M. Cao, W. Yan, and S. S. Ge, "Efficient routing for precedence-constrained package delivery for heterogeneous vehicles," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 248–260, Jan. 2020.
[41] X. Bai, W. Yan, M. Cao, and D. Xue, "Distributed multi-vehicle task assignment in a time-invariant drift field with obstacles," *IET Control Theory Appl.*, vol. 13, no. 17, pp. 2886–2893, Nov. 2019.
[42] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *Eur. J. Oper. Res.*, vol. 126, no. 1, pp. 106–130, Oct. 2000.
[43] M. Englert, H. Röglin, and B. Vöcking, "Worst case and probabilistic analysis of the 2-Opt algorithm for the TSP," *Algorithmica*, vol. 68, no. 1, pp. 190–264, Jan. 2014.
[44] D. J. Rosenkrantz, R. E. Stearns, P. M. Lewis, and II, "An analysis of several heuristics for the traveling salesman problem," *SIAM J. Comput.*, vol. 6, no. 3, pp. 563–581, Sep. 1977.
[45] F. B. Sorbelli, S. Carpin, F. Coró, S. K. Das, A. Navarra, and C. M. Pinotti, "Speeding up routing schedules on aisle graphs with single access," *IEEE Trans. Robot.*, vol. 38, no. 1, pp. 433–447, Feb. 2022.

SEUNGBEEN LEE received the B.S. degree in information and communication engineering from Soonchunhyang University, South Korea, in 2024. He is currently pursuing the M.S. degree in computer science and artificial intelligence with Dongguk University, Seoul, South Korea. His current research interests include multi-robot systems, task assignment problem, and MAPF.

CHANYOUNG LEE is currently pursuing the B.S. degree in computer science with Dongguk University, Seoul, South Korea. He is also an Undergraduate Researcher with the AI and Robotics Laboratory. His research interests include multi-robot systems, task assignment problem, and MAPF.

WONPIL YU received the B.S. degree in control and instrumental engineering from Seoul National University, Seoul, South Korea, in 1992, and the M.S. and Ph.D. degrees in electrical engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 1994 and 1999, respectively. He is with the Intelligent Robot Research Group, Electronics and Telecommunication Research Institute (ETRI), South Korea. He is also active in developing technology standards in mobile robotics, where he is currently with the Map Data Representation (MDR) Working Group with technical sponsorship from the IEEE Robotics and Automation Society. Prior to joining ETRI, in 2001, he worked for the Agency for Defense Development (ADD), Daejeon, where he was involved in the development of a precision stabilizer for a radar seeker system. His research interests include mobile robotics, agricultural robotics, and perception technology.

SOOHWAN SONG received the B.S. degree in information and communication engineering from Dongguk University, Seoul, South Korea, in 2013, and the M.S. and Ph.D. degrees in computer science from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2015 and 2020, respectively. He is currently an Assistant Professor with the College of AI Convergence, Dongguk University. Prior to joining Dongguk University, he was a Senior Researcher with the Field Robotics Research Division, Electronics and Telecommunications Research Institute (ETRI), Daejeon. His research interests include robotics, motion planning, multi-robot systems, and computer vision.